Graph Neural Network to Predict Flight Arrival

Author: Jon Montgomery

1. Introduction

Commercial air travel is highly sensitive to delays, which propagate through the network of flights and airports and impose real costs on airlines and passengers. The goal of this project is to predict flight arrival delay at the time of departure, using a deep learning model that captures both temporal dynamics and network structure in the air transportation system. Deep learning is well-suited to this setting because it can learn complex, nonlinear relationships from large volumes of historical data and exploit both the time-series nature of congestion and the graph structure of the route network. More accurate delay predictions can support better passenger information, and disruption management, and congestion handling at the arrival airport.

The data come from the Bureau of Transportation Statistics Carrier On-Time Performance data and consists of flight-level records between the top 35 airports, with each row describing a single flight. For each flight, the data includes its origin and destination airports, scheduled departure and arrival times, and realized arrival delay (the prediction target), along with engineered features that summarize the state of the network around the departure time. These features are organized so that we can view the system as a time series of airport states (nodes) and a set of directed edges for flights between airports at each time step.

To exploit this geometric structure, the model is built as a graph-based neural network with temporal components. At a high level, the architecture has three parts:

- Temporal encoder: A sequence model (transformer-style) that processes a fixed-length history of node features for each airport, capturing how conditions evolve over time.
- Graph message-passing layers: A graph neural network that operates on the airport–flight graph at the current time step, allowing information to flow along routes from origin to destination and through connecting airports.
- Prediction head: A feed-forward network that combines the encoded node and flight features to output a scalar regression prediction: the expected arrival delay (after an internal scaling step).

This design allows the model to learn both local temporal patterns (e.g., how congestion at a given airport builds up over time) and global network effects (e.g., how delays propagate along common connections), which are critical for realistic delay prediction.

2. Methodology

2.1 Graph structure

The core modeling choice in this project is to represent the air transportation system as a temporal graph. Each airport is a node, direct services between airports define edges, and individual flights are treated as edge-level events that occur at specific times.

Time is discretized into 30-minute buckets. For each airport and time bucket, I compute a node feature vector that summarizes local congestion: how many flights are arriving and departing, and what the recent delays look like. These node features form a multivariate time series per airport, which is then processed by a temporal encoder to produce a congestion embedding for each airport at the departure time of interest.

At the same time step, the static route network between airports is represented as a directed, unweighted graph. For each individual flight to be predicted, I construct a flight feature vector that includes the route distance, scheduled elapsed time, time-of-day and day-of-week encodings, carrier identity, and holiday indicator. The model first encodes airport congestion histories into node embeddings, then runs graph attention over the route network, and finally combines origin and destination embeddings with flight-level features to output a scalar delay prediction.

2.2 Features

The table below summarizes the node and flight features used in the model:

Type	Feature	Variable type / range	Description
Node	Departure Count	Integer count (0+)	Number of flights departing from the airport in a time bucket, min–max scaled to ([0, 1]).
	Arrival Count	Integer count (0+)	Number of flights arriving at the airport in a time bucket, min–max scaled to ([0, 1]).
	Mean departure delay	Float (minutes), roughly ([-60, 200+])	Average departure delay for flights departing from the airport, min–max scaled to ([0, 1]).
	Mean arrival delay	Float (minutes), roughly ([-60, 200+])	Average arrival delay for flights arriving at the airport, min–max scaled to ([0, 1]).
Flight	Scheduled elapsed time	Float (minutes), typically ([20, 400])	Scheduled gate-to-gate flight time for the specific flight, min–max scaled using training data.
	Distance	Float (miles), typically ([100, 3000+])	Route distance for the specific flight, min–max scaled using training data.
	Day of week (sin+cos)	Float, each component in ([-1, 1])	Sinusoidal encoding of day of week (cyclic feature), naturally bounded so no additional scaling.

Type	Feature	Variable type / range	Description
	Dep Time of day (sin+cos)	Float, each component in ([-1, 1])	Sinusoidal encoding of departure time of day (cyclic feature), naturally bounded so no additional scaling.
	Arr Time of day (sin+cos)	Float, each component in ([-1, 1])	Sinusoidal encoding of CRS predicted arrival time of day (cyclic feature), naturally bounded so no additional scaling.
	Carrier one-hot (15 values)	Binary indicator ({0, 1}) per carrier	One-hot encoding of the marketing carrier (15 carriers), used directly without further scaling.
	Holiday (1, 0)	Binary indicator ({0, 1})	1 if the flight is on a US holiday, 0 otherwise, used directly without further scaling.
Output	Predicted arrival delay	Float (minutes), scaled to ([-10, 10])	Target variable: arrival delay in minutes, linearly transformed to the ([-10, 10]) range using training-set percentiles and clipped at the upper end to reduce the impact of extreme outliers.

2.3 Data preparation

Before training the model, I transform the raw flight records into tensors suitable for graph-based learning. First, I filter out cancelled and diverted flights and any rows with missing arrival delay. After filtering these rows, there were no null values left in the dataset. I then split the data chronologically into train, validation, and test sets by year and month to avoid any leakage of future information into the past. All preprocessing decisions (such as normalization and target scaling) are fit on the training split only and reused for validation and test.

For node features, I discretize time into 30-minute buckets and, for each airport and bucket, compute counts of departures and arrivals as well as the mean departure and arrival delays. These four values form a 4-dimensional node state vector, arranged into a tensor of shape $[T, N, F_{\text{node}}]$ where (T) is the number of time buckets and (N) is the number of airports. I then apply channel-wise min–max normalization so that each node feature lies in $[0, 1]$, which stabilizes training and makes the temporal encoder easier to optimize.

For flight features, I build per-flight feature vectors that include scheduled elapsed time, route distance, cyclic encodings of day-of-week and time-of-day (using sine and cosine), a binary holiday indicator, and a one-hot encoding of the carrier. Numeric flight features are also min–max scaled using training-set statistics. To keep the regression target well-behaved, arrival delay in minutes is mapped to the range ([-10, 10]) using percentiles estimated from the training set; very extreme delays (the top 98th percentile) are clipped to this range to avoid loss domination from these outliers. The final outputs of

this stage are PyTorch datasets and loaders for train/validation, together with the node-state tensor, graph connectivity, and scaling parameters needed to interpret predictions back in minutes.

Without outlier clipping the distribution of delays is strongly skewed toward long delays.

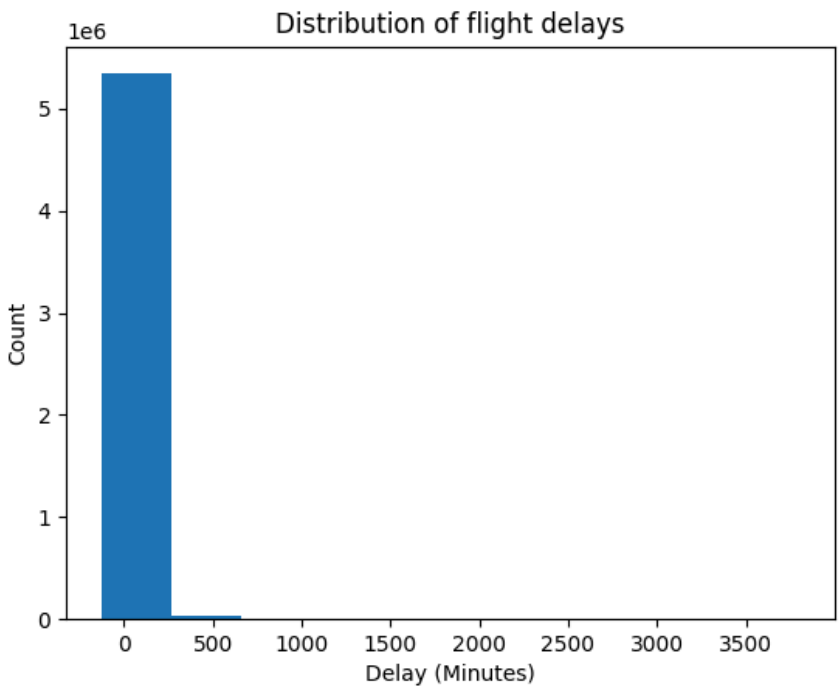

Figure 1: Distribution of unscaled flight delays

With scaling, clipped at the 98th percentile, the distribution is still skewed toward long delays but maintains resolution for the normal range. The 98th percentile is 160 minutes, so the flights with longer delays can be classified as “long delay flights” and can be handled more accurately by targeted systems.

After processing, the distribution of flight delays is shown in Figure 2.

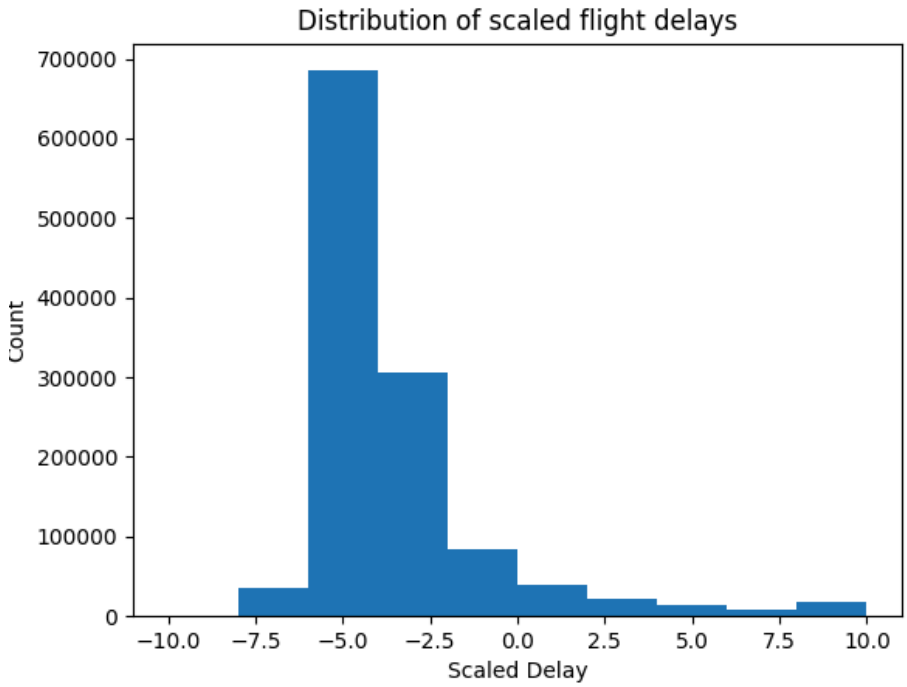

Figure 2: Distribution of scaled flight delays

3. Training

The model is trained to predict arrival delay in the scaled ([-10, 10]) space using mean squared error as the loss metric.

I use the AdamW optimizer with weight decay for regularization. Gradients are clipped to a maximum norm of 1.0 to stabilize training. Mixed-precision (FP16) training is enabled on GPU to reduce memory use and speed up computation. Batches are formed by grouping flights that share the same departure time bucket, which allows efficient reuse of the graph and state tensors across samples in a batch.

Training runs for a fixed number of epochs (or fewer if early stopping triggers). Early stopping is applied when validation loss increases for two consecutive epochs; at that point training halts and the model weights from the best validation epoch are restored. This prevents overfitting when the model has sufficient capacity.

3.1 Hardware

Training and architecture search were distributed across four NVIDIA GeForce RTX 2080 Ti GPUs. Each GPU ran an independent training session for a single configuration, allowing multiple configurations to be evaluated in parallel. With this setup, I trained four models simultaneously, with a typical training model training run taking about 5 hours to complete per configuration.

3.2 Grid search

To identify a suitable architecture for this dataset, I ran a grid search over model structure and size. The search varies the temporal and graph encoder dimensions (e.g. `d_model`, number of attention heads, number of layers) and the size of the prediction head, while keeping learning rate, dropout, weight decay, and batch size fixed. Runs are distributed across available GPUs (one training job per GPU in parallel), and validation loss is recorded for each configuration. When a larger `d_model` yields worse validation loss than a smaller one for the same other hyperparameters, the search skips further increases in that dimension for that configuration, which reduces the number of runs without losing good candidates.

The purpose of this grid search is to find the general structure and size of a model that performs well on the data—that is, to narrow down the right order of magnitude for capacity (e.g. 64 vs 128 vs 256 dimensions) and depth, rather than to produce a fully tuned model. Once a promising model size and complexity have been established, the intention is to hone the hyperparameters further (e.g. learning rate, regularization, number of epochs) and to address underfitting or overfitting in a targeted way, using the grid results as a starting point for more focused experimentation.

3.3 Further tuning

After the grid search identified a good range of model dimensionalities, I performed targeted hyperparameter tuning to refine performance. In this phase I adjusted batch size, learning rate, and dropout around the best architecture from the grid, monitoring both training and validation loss to diagnose underfitting or overfitting. Tuning all features simultaneously would be far too computationally intensive and time consuming.

3.4 Null model

I considered two null models as baselines. In the first, we predict no delay (zero) for every flight. In the second, we predict the mean delay observed in the training data for every flight. The zero-delay null model had a validation loss of 0.9964 and a mean absolute error of 65.8457 minutes. The mean-delay null model (training mean = 8.1 minutes) had a validation loss of 0.3368 and an MAE of 27.18 minutes. Since the mean delay model is better in both dimensions, I used it as the baseline to justify the GNN's added complexity.

4 Results

4.1 Model architecture

The grid search evaluated 24 configurations over `d_model` (8, 32, 128), `n_heads` (2, 4), `graph_n_layers` (2, 3), and `head_hidden` (64, 256), with temporal layers fixed at 2. The results of the grid search are displayed in Figure 3, arranged primarily by `d_model`. All models in the grid search were trained with dropout=0.2, batch size=16384, learning rate=3×10^{-4}, and weight decay=0.01.

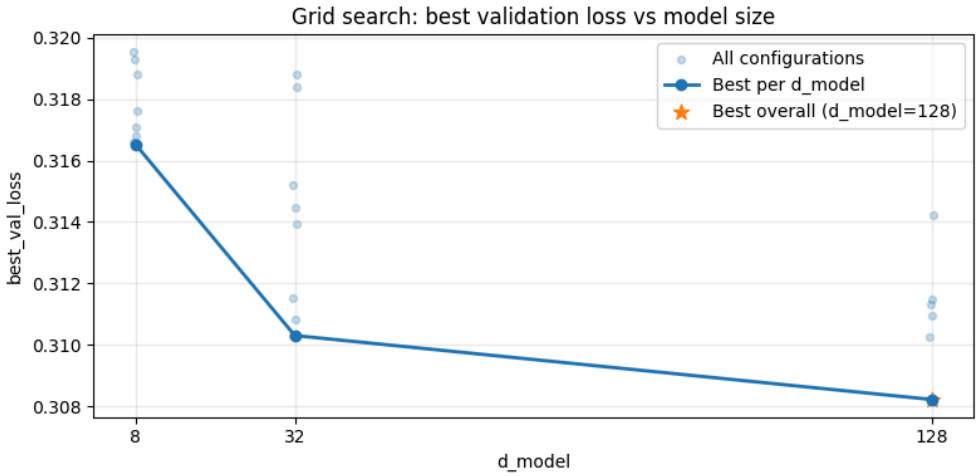

Figure 3: Grid Search

The best-performing configuration from the grid search used a model size d_model = 128, with n_heads = 4, temporal_n_layers = 2, graph_n_layers = 2, and a prediction head hidden size of 256. This configuration achieved a validation loss of 0.3082 in the scaled ([-10, 10]) and MAE of 20.62. In both metrics, it improves on both null models: it outperforms the zero-delay baseline (loss 0.9964, MAE 65.85 minutes) and the stronger mean-delay baseline (loss 0.3368, MAE 27.18 minutes), demonstrating that the GNN architecture provides a meaningful gain over simple constant predictors.

The training curve for the best model in the grid search is shown in Figure 4.

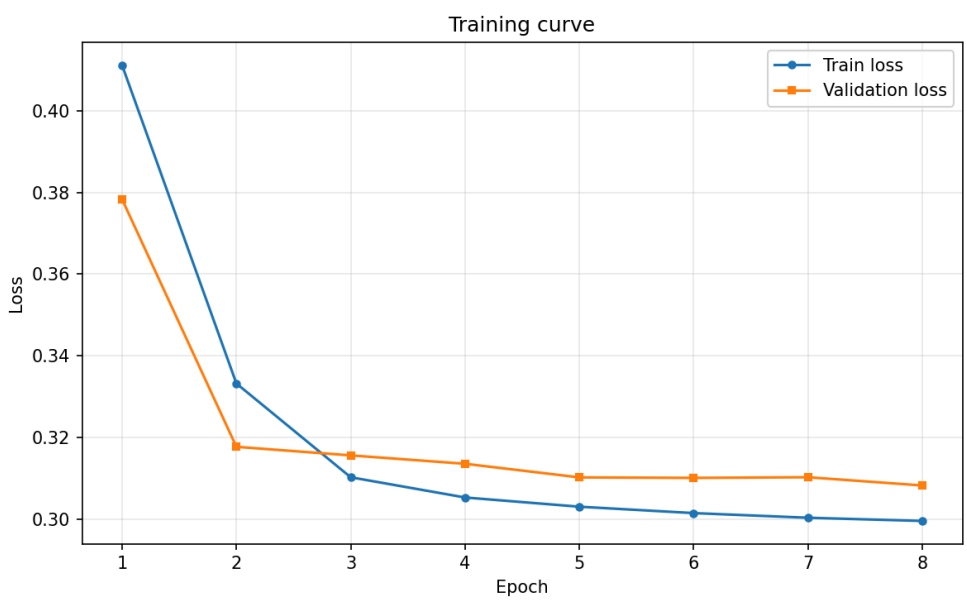

Figure 4: Best model architecture

The training curve indicates that the model had not overfit, and running for more epochs could yield better results. The model is also among the most complex that I trained, indicating that the graph model benefits from its complexity. If I had more time, I would explore larger model architectures.

4.2 Model Refinement

After selecting a model shape, I have concluded that the model has underfit, and could benefit from longer training. I will also introduce two edge features, route distance, and estimated flight time that I neglected in earlier models. The edge features should improve the power of the model in the congestion propagation phase.

I ran the selected architecture with the two additional edge features to see if the additional information for the GAT improves the power of the model. The model with additional edge features achieved the best validation loss so far, and started to overfit at epoch 10.

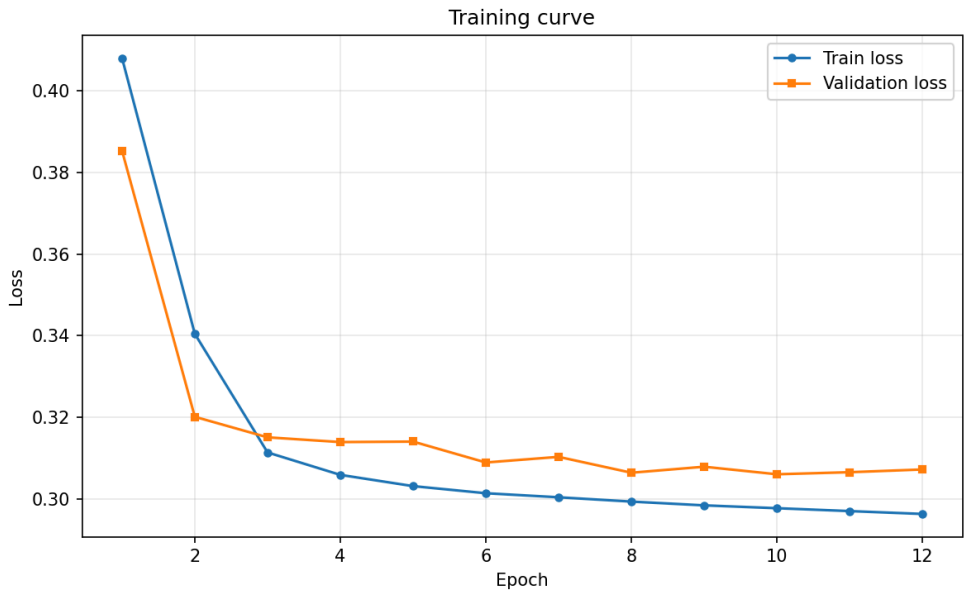

Figure 5: Best model with edge features

To address the overfitting, I decreased the batch size to 8192, and found a small performance benefit with a longer, smoother training curve. The best loss was achieved for this model at epoch 19.

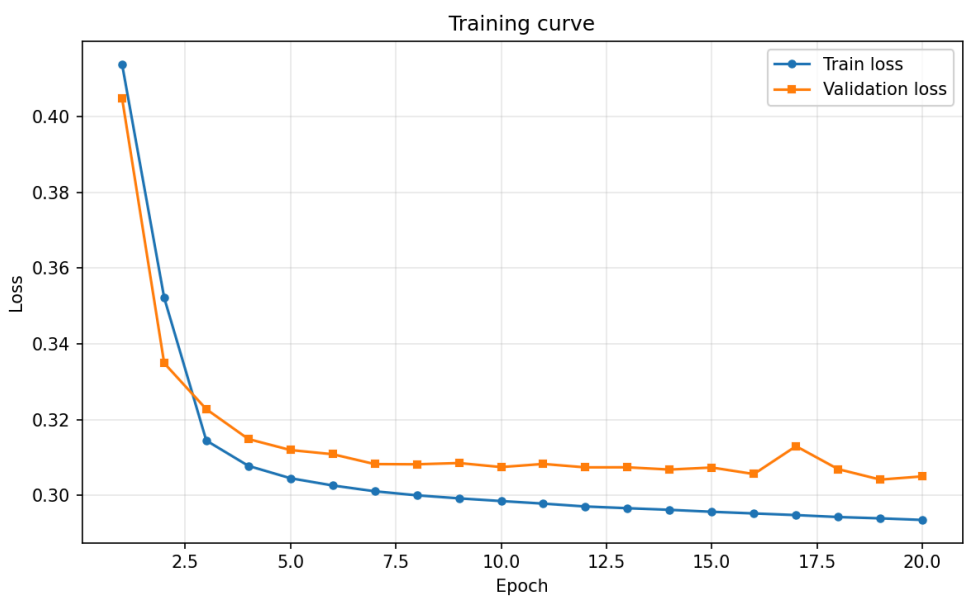

Figure 6: Best model with smaller batch size

The results of the best performing models is shown in Figure 7. You can see that the models with edge features achieve slightly lower validation loss, but have slightly higher MAE values. This occurs because mean squared error penalizes large residuals quadratically, so the refined models have focused on reducing a small number of very large mistakes (improving MSE), while slightly increasing typical absolute errors; mean absolute error, which weights all deviations linearly, is therefore a bit worse even though the tail of the error distribution has improved.

Model	MSE	MAE (Minutes)
No Edge Features	0.3082	20.62
With Edge Features	0.3060	20.78
Smaller Batch Size	0.3042	20.90
Null (mean prediction)	0.3368	27.18

Figure 7: Model metric evaluation

5. Conclusion and future work

This project demonstrates that a spatiotemporal graph neural network can meaningfully outperform simple null models for flight arrival delay prediction, and that performance improves as the model gains capacity in both the temporal transformer and the prediction head. My experiments suggest that the task benefits from additional complexity and scale, particularly in how the temporal encoder captures long-range patterns and how the head combines node and flight features. As a next step, I would continue exploring larger graph models (more layers, higher dimensional embeddings, and wider heads), and I would like to incorporate weather information into the graph so that local and route-level meteorological conditions can directly influence the learned representations. If I were to deliver this as a production system, I would invest in orchestrating these richer data sources and rigorously evaluating very large models, trading off predictive gains against serving cost and latency.